

Usando *metaclasses*

No Python, classes também são objetos, e alguém as cria: a **metaclasse**. A padrão é `type`. Escrevendo a sua, você executa código no momento em que uma classe é **definida**: registrar, validar, acrescentar atributos ou controlar a criação de instâncias. Nesta aula: classes como objetos, `type()` criando classes, metaclasses com `__new__` e `__call__`, e o `__init_subclass__`, que resolve a maioria dos casos sem metaclasse.

Classe é objeto

`type(nome, bases, dict)`

`metaclass=`

`__new__` · `__call__`

Registro · validação

`__init_subclass__`

Formação Claude Code 2026 · Baseado em docs.python.org (Data model: Metaclasses, Customizing class creation, `__init_subclass__`; Built-in Functions: `type`), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

Quem cria *a classe*?

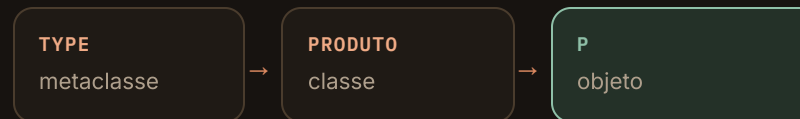
tudo_objeto.py

```
class Produto:
    pass

p = Produto()
print(type(p))          # o objeto foi criado pela classe
print(type(Produto))    # a classe foi criada por... type
print(type(int), type(str))
print(type(type))       # type é a metaclasses padrão
```

SAÍDA

```
<class '__main__.Produto'>
<class 'type'>
<class 'type'> <class 'type'>
<class 'type'>
```



- ✓ Objetos são criados por classes; classes são criadas por uma **metaclasses**
- ✓ A metaclasses de quase tudo é `type`, inclusive de `int` e `str`
- ✓ `type` é metaclasses de si mesma

Criando uma classe *sem o class*

com_type.py

```
def descrever(self):
    return f"Ponto({self.x}, {self.y})"

# type(nome, bases, atributos) cria uma classe na hora
Ponto = type("Ponto", (), {"x": 0, "y": 0,
                           "descrever": descrever})

p = Ponto()
p.x = 3
print(p.descrever())
print(Ponto.__name__, type(Ponto))
```

SAÍDA

```
Ponto(3, 0)
Ponto <class 'type'>
```

- ✓ Com três argumentos, `type(nome, bases, atributos)` monta uma classe
- ✓ É exatamente o que o Python faz quando lê um bloco `class`
- ✓ `bases` é a tupla das classes-mãe; o dicionário vira atributos e métodos

Na prática, você escreve `class`. Ver o `type()` por dentro explica o que a metaclasses recebe.

Código que roda *na definição*

primeira_meta.py

```
class Rastreada(type):
    def __new__(mcs, nome, bases, atributos):
        print(f"criando a classe {nome}...")
        atributos["origem"] = "Rastreada"      # injeta atributo
        return super().__new__(mcs, nome, bases, atributos)

class Pedido(metaclass=Rastreada):            # roda aqui!
    pass

print("depois do class")
print(Pedido.origem)
print(type(Pedido).__name__)
```

SAÍDA

```
criando a classe Pedido...
depois do class
Rastreada
Rastreada
```

- ✓ Uma metaclasses é uma classe que herda de `type`
- ✓ `__new__` recebe o nome, as bases e o dicionário de atributos **antes** de a classe existir
- ✓ "criando a classe" aparece no bloco `class`, sem nenhum objeto criado
- ✓ `metaclass=Rastreada` escolhe a metaclasses

Plugins que *se registram sozinhos*

registro.py

```
class RegistraPlugins(type):
    plugins = {}

    def __new__(mcs, nome, bases, atributos):
        classe = super().__new__(mcs, nome, bases, atributos)
        if bases:                                # não registra a base
            mcs.plugins[nome.lower()] = classe
        return classe

class Plugin(metaclass=RegistraPlugins):
    pass

class Pdf(Plugin):
    pass

class Csv(Plugin):
    pass

print(list(RegistraPlugins.plugins))
leitor = RegistraPlugins.plugins["csv"]()      # cria pelo nome
print(type(leitor).__name__)
```

SAÍDA

```
['pdf', 'csv']
Csv
```

- ✓ Toda subclasse de `Plugin` entra no dicionário assim que é definida
- ✓ Ninguém precisa lembrar de registrar a classe nova
- ✓ Depois, cria-se o objeto pelo nome: útil para leitores de formatos, comandos, rotas
- ✓ `if bases` deixa de fora a classe base, que não tem mãe

Recusar classes *incompletas*

validacao.py

```
class ExigeExecutar(type):
    def __new__(mcs, nome, bases, atributos):
        if bases and "executar" not in atributos:
            raise TypeError(f"{nome} precisa de executar()")
        return super().__new__(mcs, nome, bases, atributos)

class Tarefa(metaclass=ExigeExecutar):
    pass

class Backup(Tarefa):
    def executar(self):
        return "backup feito"

print(Backup().executar())

class Relatorio(Tarefa):      # esqueceu o executar()
    pass
```

SAÍDA

```
backup feito
Traceback (most recent call last):
  File "validacao.py", line 20, in <module>
    class Relatorio(Tarefa):      # esqueceu o executar()
        pass
  File "validacao.py", line 4, in __new__
    raise TypeError(f"{nome} precisa de executar()")
TypeError: Relatorio precisa de executar()
```

- ✓ A metaclasses confere o dicionário de atributos e lança `TypeError` se faltar `executar`
- ✓ O erro aparece ao **definir** `Relatorio`, antes de qualquer uso
- ✓ Erros de estrutura descobertos cedo, quando o módulo é importado

Para exigir métodos, o módulo `abc` (classes abstratas) é a solução padrão; aqui é para ver como funciona por dentro.

Controlando a criação: *singleton*

singleton.py

```
class Unica(type):
    _instancias = {}

    def __call__(cls, *args, **kwargs):    # roda em Config()
        if cls not in cls._instancias:
            nova = super().__call__(*args, **kwargs)
            cls._instancias[cls] = nova
        return cls._instancias[cls]

class Config(metaclass=Unica):
    def __init__(self):
        print("carregando configurações...")

a = Config()
b = Config()
print(a is b)
```

SAÍDA

```
carregando configurações...
True
```

- ✓ `Config()` chama o `__call__` da metaclasses, que normalmente cria o objeto
- ✓ Aqui ele guarda a primeira instância e devolve sempre a mesma
- ✓ "carregando" aparece uma vez só, e `a is b` é `True`

Use com moderação: um objeto único e global dificulta testes. Muitas vezes, um módulo com variáveis resolve.

`__init_subclass__`: *sem metaclasse*

`init_subclass.py`

```
class Plugin:
    plugins = {}

    def __init_subclass__(cls, **kwargs):  # sem metaclasse
        super().__init_subclass__(**kwargs)
        Plugin.plugins[cls.__name__.lower()] = cls

class Pdf(Plugin):
    pass

class Csv(Plugin):
    pass

print(list(Plugin.plugins))
```

SAÍDA

```
['pdf', 'csv']
```

- ✓ O mesmo registro da seção 04, com um método comum na classe base
- ✓ `__init_subclass__` roda toda vez que uma subclasse é criada
- ✓ Sem metaclasse, sem conflito com outras metaclasses

Regra prática: tente primeiro `__init_subclass__`, decoradores de classe ou `abc`. Metaclasse é para quando nada disso basta.

Toda subclasse *com versão*

Enunciado: faça a classe `Modulo` recusar qualquer subclasse sem o atributo `versao` no formato `"X.Y.Z"`.

Uma resposta possível

```
class Modulo:
    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        versao = str(getattr(cls, "versao", ""))
        if versao.count(".") != 2:
            raise TypeError(f"{cls.__name__}: versao 'X.Y.Z'")

class Pagamentos(Modulo):
    versao = "2.1.0"

print("Pagamentos ok:", Pagamentos.versao)
try:
    class Estoque(Modulo):
        versao = "2"
except TypeError as erro:
    print("Erro:", erro)
```

SAÍDA

```
Pagamentos ok: 2.1.0
Erro: Estoque: versao 'X.Y.Z'
```

- ✓ A checagem roda ao definir a classe `Estoque`, por isso o `try` envolve o `class`
- ✓ Desafio: valide o formato com o regex `r"\d+\.\d+\.\d+"` (aula 06)

Teste o que *ficou*

1. `type(int)` devolve:

A) `int`

B) `<class 'type'>`

C) `object`

D) Erro

B.

2. O `__new__` de uma metaclasses roda quando:

A) Um objeto é criado

B) A classe é definida

C) Um método é chamado

D) O programa termina

B.

3. Para registrar subclasses automaticamente, o mais simples é:

A) `__init_subclass__`

B) Uma metaclasses

C) Um decorador de função

D) Uma variável global

A.

4. No singleton, qual método da metaclasses intercepta `Config()`?

A) `__new__`

B) `__init__`

C) `__call__`

D) `__repr__`

C.

A classe

das classes

Classes são objetos criados por uma metaclasses, normalmente `type`. Uma metaclasses sua (herdando de `type`) roda código na definição da classe com `__new__` (registrar, validar, injetar atributos) e na criação de objetos com `__call__`. Antes de recorrer a ela, veja se `__init_subclass__`, um decorador de classe ou `abc` resolvem.

`type`

`metaclass=`

`__new__`

`__call__`

`__init_subclass__`

Próxima aula → 13 · Programação Funcional

Fontes: docs.python.org/3/reference/datamodel.html#metaclasses; docs.python.org/3/reference/datamodel.html#customizing-class-creation (`__init_subclass__`); docs.python.org/3/library/functions.html#type; docs.python.org/3/library/abc.html, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.